



# Monolith vs Microservices

Week 4 • Day 1 • Technical Architecture & Constraints

TPM Academy



# Day 1 Objectives

- Frame mono-vs-micro decisions in **business + operational terms**, not framework hype
- Apply the **three-question frame**: deploy cadence, failure domain, scaling axis
- Recognize the "**majestic monolith**" as the often-correct default
- Draft **Section 1 of the Technical Concept Document (TCD)** — your architecture stance with trade-off and revisit trigger
- Begin **Section 2 — Integration map** with failure-handling stances



# Today's Shape

| Clock         | Block                       | Outcome                      |
|---------------|-----------------------------|------------------------------|
| 09:00 – 09:15 | Opening                     | AI restored; PRD on the wall |
| 09:15 – 10:30 | Block 1 + Activity 1        | Mono/Micro Triage            |
| 10:45 – 12:00 | Block 2 + Activity 2        | Apply the frame to your PRD  |
| 13:00 – 14:15 | Block 3 + Activity 3        | Integration map (first pass) |
| 14:30 – 16:00 | Block 4 + Activity 4 + Wrap | AI-assisted architecture Q&A |

# Block 1 — The Vocabulary & the Frame

Enough to follow the architect's reasoning, not enough to win arguments by quoting taxonomy



# The Vocabulary Card

| Term                        | What it means                                 | TPM signal                              |
|-----------------------------|-----------------------------------------------|-----------------------------------------|
| <b>Monolith</b>             | One deployable, one repo, one runtime         | Faster start; harder team scaling       |
| <b>Modular monolith</b>     | Monolith with strict module boundaries        | Often the right answer                  |
| <b>Microservice</b>         | Independently deployable, owned by one team   | Independent scale & failure domains     |
| <b>SOA</b>                  | Coarse-grained services, often shared infra   | What "microservices" often actually is  |
| <b>Distributed monolith</b> | Multi-deploy that <i>must</i> deploy together | Anti-pattern                            |
| <b>FaaS</b>                 | Per-function deploys                          | Bursty workloads; not a default         |
| <b>Event-driven</b>         | Components via queues / events                | Decouples timing; complicates debugging |

# ? The Three-Question Frame

## 1. Deploy cadence

*Whose deployment cadence does separation protect?*

If only one team owns it, separation buys mostly ops cost.

## 2. Failure domain

*What independent failure do we want?*

If the feature can't function without its dependencies anyway, separation buys no resilience.

## 3. Scaling axis

*What scale curve are we anticipating?*

If load is uniform, one runtime is cheaper.

**Heuristic:** 2+ "yes with evidence" → microservice plausible. 2+ "we don't know" → modular monolith.



# The "Majestic Monolith" Position

Most early-stage features ship better as **modules in a well-factored monolith** than as new services.

- **Faster end-to-end testing** (one process, one deploy)
- **Lower observability complexity** (no inter-service tracing tax)
- **Lower coordination cost** (no shared API contract)
- **Reversibility** — extract a module to a service later; merging two services is hard

**The TPM job:** surface the business cost of *premature* service-orientation. Not to win the debate — to set the right starting point.

# Activity 1 — Mono/Micro Triage

Triads • 35 minutes

8 anonymized examples. Apply the three-question frame; stamp Monolith / Micro / Modular monolith / Need more info.

- 15 min: stamp all 8
- 10 min: pick 2 strongest cases for separation, 2 for staying together
- 10 min: identify 1 disagreement to share



15 stamp • 10 pick • 10 disagree



# Block 2 — Apply the Frame to Your PRD

Draft TCD Section 1 — Architecture stance



# The Stance Template

```
## 1. Architecture stance
```

```
<p>We propose this feature ship as <strong><modular monolith / new module  
in the existing service / new microservice / hybrid></strong> because:</p>  
<ul>  
<li><strong>Deploy cadence:</strong> [whose tempo are we protecting]</li>  
<li><strong>Failure domain:</strong> [what should keep running if X fails]</li>  
<li><strong>Scaling axis:</strong> [what curve do we expect]</li>  
</ul>
```

```
<strong>Trade-off accepted:</strong> [the cost of this choice – be honest]  
<strong>Revisit if:</strong> [the trigger that would change the answer]
```

**"Revisit if"** is the senior-author signal. A concrete trigger ("if our deploy frequency drops below daily" or "if traffic to this feature grows 10x") beats "later."



# Worked Stance (FieldPulse reconcile)

## ## 1. Architecture stance

```
<p>We propose this feature ship as a <strong>new module in the existing
mobile-app monolith</strong> because:</p>
<ul>
<li><strong>Deploy cadence:</strong> Only the Dispatcher Workflow Squad will own
this code; we have no cross-team deploy contention to resolve.</li>
<li><strong>Failure domain:</strong> Reconcile depends on Tickets API and Auth;
if either is down, the feature can't function. Separation buys
no resilience.</li>
<li><strong>Scaling axis:</strong> Reconcile traffic is bounded by dispatcher
shifts (predictable daily curve, < 10K active dispatchers).
Same scale curve as the rest of the mobile app.</li>
</ul>
<p><strong>Trade-off accepted:</strong> A future split (e.g., into a "shift-end
workflows" service) becomes harder once this code lives next to
existing modules.</p>
```

```
<strong>Revisit if:</strong> Reconcile traffic exceeds 5x rest-of-app, OR a
second team starts contributing reconcile features.
```

## Activity 2 — Apply the Frame

Triads • 40 minutes

Run the three-question frame against your locked Product Requirements Document (PRD) feature. Draft TCD Section 1.

- 5 min: re-read PRD Section 10 (Dependencies)
- 15 min: 3-question pass; honest answers with evidence
- 15 min: draft Section 1
- 5 min: sanity check — real evidence or hand-wave?



5 + 15 + 15 + 5

# Block 3 — Integration Map (First Pass)

What the feature touches; how it fails



# The Integration Table

| System            | Owner    | Sync/Async | R/W | Failure handling    |
|-------------------|----------|------------|-----|---------------------|
| Auth (SSO)        | Identity | Sync       | R   | Fail closed         |
| Audit event store | Platform | Async      | W   | Queue + DLQ         |
| Tickets API       | Dispatch | Sync       | RW  | Retry x3, then fail |
| Notification svc  | Comms    | Async      | W   | Best-effort, drop   |

Each integration carries a failure stance — e.g., single sign-on (SSO) for auth, a dead-letter queue (DLQ) for audit writes — which later hardens into acceptance criteria (AC).

**"Failure handling: TBD"** is fine if it's an open question. **"Hopefully it works"** is not.

# Two-Way Contracts

Some integrations require **coordination with another team** — they have to change something for you, sometimes touching personally identifiable information (PII) handling. Mark these explicitly.

| Contract     | What we need                 | What they need                   |
|--------------|------------------------------|----------------------------------|
| Tickets API  | New endpoint for batch close | Schema review; rate-limit budget |
| SSO scopes   | New scope `reconcile.write`  | Security review; rollout plan    |
| Audit schema | 7 new event types            | Schema versioning; PII review    |

Two-way contracts go on the **Stakeholder + Sign-off matrix** (TCD Section 6, Friday).

## Activity 3 — Integration Map

Triads • 40 minutes

List integrations from PRD Section 10 + anything missed. Characterize each.  
Mark two-way contracts.

- 15 min: list integrations
- 15 min: characterize sync/async, R/W, failure
- 10 min: identify two-way contracts



15 + 15 + 10



# Block 4 — AI is Back

Stress-test the stance; surface what we forgot



# AI Norms (Refresher)

- Use Week 1 prompt patterns: **Role / Context / Constraints / Format**
- Use Week 2 Day 4 discipline: **provenance log** for any AI-assisted output
- Stay on the safe side of the bright line: **structure source material; do not ask AI for facts about your real systems**
- Cross-validate: AI surfaces possibilities; you decide what's real

**Not allowed:** "AI told me our system uses X." That's hallucination. AI doesn't know your codebase.



# Today's Two Prompts

## A. Stress-test our stance

Paste TCD Section 1 + integration table. Ask for the 3 strongest objections, each with the specific scenario.

## B. What did we forget?

Paste integration table. Ask: "what categories of dependency are commonly forgotten for a feature with this footprint?"

```
Role: Senior staff engineer reviewing an architecture stance.  
Context: <paste tcd="" section="" 1="" += "" integration="" table="">  
Task: Identify the 3 strongest objections to the stance.  
Constraints:  
- Treat the proposed approach as a starting point, not a verdict  
- For each objection, name the specific scenario where it would matter  
- Flag any place the rationale relies on assumptions not stated above  
Format: Numbered list of objections, each with a scenario.</paste>
```

# Activity 4 — AI-Assisted Architecture Q&A

Triads • 45 minutes • Wrap

Run Prompt A and Prompt B. Adopt / defer / reject each suggestion. Update Section 1 + Section 2. Add provenance note.

- 10 min: Prompt A → 3 objections
- 10 min: adopt / defer / reject decisions
- 10 min: Prompt B → missing dependencies
- 10 min: update Sections 1–2 + provenance note
- 5 min: AI-prose check; rewrite if generic



10 + 10 + 10 + 10 + 5 · 15-min wrap



# Day 1 Wrap

## What we built

- Three-question frame (deploy / failure / scale)
- Architecture stance with revisit trigger
- Integration table with failure handling
- Two-way contracts list
- AI provenance discipline restored

## What opens tomorrow

- **System security & compliance**
- Threat-model pass using the STRIDE model (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege)
- Revising PRD's Security non-functional requirements (NFRs) at architectural level

**Bring to Day 2:** Your TCD Sections 1–2. We'll thread the security model through them tomorrow.

# Questions & Reflection

If your stance is wrong, which assumption fails first?